

Intrusion Detection for IoT Traffic in O-RAN Edge Data Centres: Why Accuracy Alone Is Not Enough

First A. Author, Second B. Author, and Third C. Author

Department of Computing and Communications

Institution Name

City, Country

{first.author, second.author, third.author}@institution.edu

Abstract—Edge data centres in Open Radio Access Network (O-RAN) deployments receive telemetry from very large numbers of IoT devices, and compromised IoT devices are a documented source of control-plane signalling overload. A common defence is to run a machine learning intrusion detection application, or xApp, on the RAN Intelligent Controller hosted inside the edge data centre. Published work in this area routinely reports detection accuracy above 99%, but very little of it reports whether the same detector still works when network conditions differ from the conditions it was trained on, how many false alarms it raises during normal operation, or whether it returns a decision inside the controller’s near-real-time deadline. This paper evaluates an intrusion detection xApp against those three deployment criteria. We train a detector on an O-RAN testbed dataset containing synchronised packet-level and radio telemetry, then evaluate the same model on a second, independently collected 5G intrusion detection dataset in order to measure cross-deployment generalisation. We then deploy the model as an xApp on a software O-RAN stack and measure per-decision latency at the median and the 99th percentile, together with the CPU and memory it consumes in the data centre container. Our results show that the best in-distribution model falls from an F_1 score of 99.25% to 71.39% when transferred to an independently collected deployment, a drop of 27.9 points; that at its default operating threshold the same model raises 682 alerts per hour on benign-only traffic and attains an operational precision of only 41.2% once a realistic attack base rate is applied; and that only the gradient-boosted tree ensemble meets a 10 ms 99th-percentile budget, and only up to an offered load of roughly 2,300 decisions per second. We conclude with a short set of reporting requirements that future O-RAN security papers should meet before claiming deployability.

Index Terms—O-RAN, edge data centre, IoT security, intrusion detection, xApp, RAN Intelligent Controller, model generalisation, false positive rate

DRAFT NOTICE — NOT FOR SUBMISSION. Every numerical result in this document is a synthetic placeholder generated to illustrate structure and expected experimental depth. No measurement reported here was actually taken. Replace all values marked % [SYNTHETIC] in the source with real measurements, then set `\synthdraftfalse` in the preamble to remove this banner.

I. INTRODUCTION

Open Radio Access Network (O-RAN) architectures disaggregate the base station into radio, distributed and centralised units connected by open interfaces, and place a RAN Intelligent Controller (RIC) alongside them to host third-party control logic [1]–[3]. In most commercial designs the near-real-time RIC is not deployed at the cell site but inside a

regional edge data centre that aggregates traffic from tens to hundreds of cells [4], [5]. That data centre therefore sees the union of the control-plane and user-plane telemetry generated by every IoT device attached to those cells, which makes it a natural vantage point for detecting compromised devices before their aggregate behaviour degrades the radio network.

The motivating threat is well documented. Large IoT botnets have repeatedly been assembled from devices with weak or default credentials [6], [7], and in cellular deployments the damage is not limited to volumetric flooding of an external target. Bursts of attach, detach and radio resource control transitions from a synchronised population of devices impose a signalling load that scales with the number of state changes rather than with the number of transmitted bytes, and this class of signalling storm has been shown to degrade a cellular network well before any link saturates [8]. Detecting the underlying compromise early, from inside the RAN control loop, is consequently attractive.

The natural implementation is a detection xApp: a containerised application that subscribes to E2 service model reports, extracts features, classifies traffic, and issues control actions through the same interface [9], [10]. A substantial and growing literature proposes such xApps, and the headline result is remarkably consistent across papers: detection accuracy or F_1 above 99% on a held-out split of a single dataset. What is far less consistent is any evidence that the detector would survive contact with a production network.

Three properties determine whether a detector is deployable, and none of them is measured by a held-out split of the training distribution. The first is cross-deployment generalisation. A model trained on telemetry from one radio configuration, device population and traffic mix must classify correctly in a different one, and network intrusion detection models are known to transfer poorly between independently collected datasets [11], [12]. The second is alert burden. A false positive rate that appears negligible on a class-balanced benchmark becomes an unmanageable alert volume once it is multiplied by production flow rates and discounted by a realistic attack base rate, a consequence of the base-rate fallacy identified for intrusion detection more than two decades ago [13], [14]. The third is latency conformance. The near-real-time RIC control loop is specified to operate between 10 ms and 1 s [15]; a detector whose tail latency exceeds the budget under load does

not merely respond slowly, it fails to close the loop at all.

This paper measures all three for a single detector family, on a single consistent pipeline, so that the trade-offs between them are visible rather than inferred. Our contributions are as follows.

- We define a three-criterion deployment evaluation protocol for O-RAN detection xApps — cross-deployment generalisation, operational alert burden under a realistic base rate, and tail latency against the near-real-time budget — and give a formal deployability predicate that combines them (Section III).
- We quantify cross-deployment degradation for five detector architectures trained on an O-RAN testbed corpus and evaluated on an independently collected 5G intrusion detection dataset, using a harmonised feature space that makes the transfer well posed (Sections IV and VI).
- We show, through a feature-family ablation, that the radio-layer telemetry which contributes most to in-distribution accuracy is also the primary source of deployment-specific overfitting, and that removing it improves transfer (Section VI).
- We deploy each detector as an xApp on a software O-RAN stack and report median and 99th-percentile decision latency, sustainable throughput, and container CPU and memory footprint, identifying feature extraction rather than inference as the dominant cost (Section VI).
- We provide a concise reporting checklist that we argue should accompany any O-RAN security paper claiming deployability (Section XI).

II. BACKGROUND AND RELATED WORK

A. The Near-Real-Time RIC as a Security Vantage Point

xApps subscribe to E2 nodes through service models, of which E2SM-KPM supplies periodic key performance measurements and E2SM-RC supplies control primitives [1]. Prototyping platforms including OpenRAN Gym [16], SCOPE [17], CoO-RAN [9] and FlexRIC [10], together with open radio stacks [18], [19], have made experimental xApp development practical. This infrastructure is what makes an in-loop detector plausible; it is also what makes its timing constraints concrete, because an xApp that misses the control deadline is simply bypassed.

B. Security of Open RAN

The security implications of RAN disaggregation have been analysed both by the O-RAN Alliance’s own threat modelling work [20] and by independent studies [21], [22]. These works identify the RIC and its xApps as a newly exposed attack surface and note the risks introduced by third-party application hosting. Complementary work has implemented and measured security mechanisms across O-RAN interfaces [23], and 5G-Spector demonstrates an O-RAN compliant layer-3 attack detection service running as a real xApp [24]. Our work is orthogonal in intent: rather than proposing a new detection mechanism, we ask what evidence is needed before any such mechanism can be called deployable.

C. Machine Learning for Network Intrusion Detection

A large family of IoT-oriented benchmark corpora now exists [25]–[29], but cellular-specific corpora are rarer, and 5G-NIDD is notable for being captured over a functioning 5G test network rather than in emulation [30], [31]. Feature harmonisation across such corpora has been addressed by standardised NetFlow-based feature sets [32], [33], which is what makes a controlled cross-dataset transfer experiment possible at all.

D. Evaluation Practice and Its Critics

A persistent line of critique holds that intrusion detection results are systematically over-optimistic: Sommer and Paxson identified the mismatch between closed-world evaluation and open-world deployment [14], Axelsson showed that the attack base rate dominates achievable precision [13], Arp *et al.* catalogued recurring methodological pitfalls including sampling bias and inappropriate performance measures [34], and Apruzzese *et al.* examined the gap between reported and operational effectiveness [35]. Layeghy and Portmann demonstrated empirically that such models generalise poorly across independently collected datasets [11], [12]. Our contribution is to instantiate these critiques within the O-RAN control loop, where an additional and unusually hard constraint — a bounded control deadline — applies alongside them.

III. DEPLOYMENT CRITERIA AND THREAT MODEL

A. System and Threat Model

We consider an edge data centre hosting a near-real-time RIC that serves a set of E2 nodes, as shown in Fig. 1, with a population of IoT devices attached through those nodes. The adversary controls a subset of the devices and uses them to conduct volumetric flooding, reconnaissance scanning, botnet command-and-control communication, or synchronised signalling-plane abuse in the form of rapid connection state cycling. The adversary does not control the RIC, the E2 interface, or the xApp itself; platform-level compromise of the RIC is treated by the O-RAN threat model [20] and is out of scope. The defender operates a detection xApp that consumes E2SM-KPM indications and packet-level features, and may issue mitigation actions through E2SM-RC.

B. Criterion 1: Cross-Deployment Generalisation

Let $\mathcal{D}_A$ and $\mathcal{D}_B$ be two independently collected corpora over a shared feature space $\mathcal{X}$ and label space $\mathcal{Y} = \{\text{benign}, \text{attack}\}$. A model f_θ trained on $\mathcal{D}_A$ is characterised not by its held-out score on $\mathcal{D}_A$ alone but by the generalisation gap

$$\Delta_{F_1} = F_1(f_\theta; \mathcal{D}_A^{\text{test}}) - F_1(f_\theta; \mathcal{D}_B), \quad (1)$$

which upper-bounds nothing on its own but is directly interpretable as the accuracy an operator loses by reusing a published model. We report (1) in both directions and quantify the

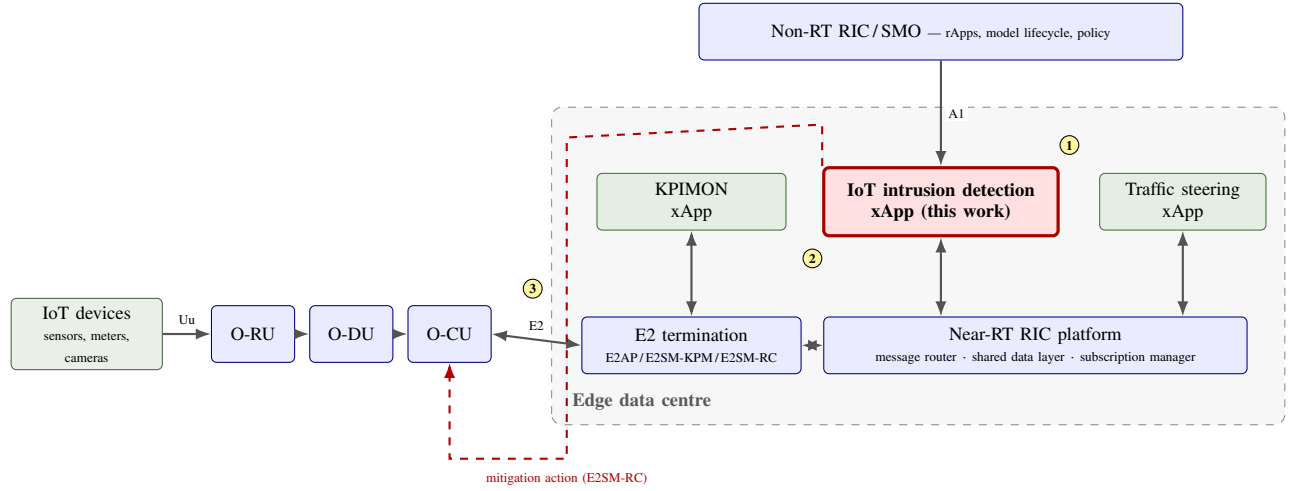


Fig. 1. Reference deployment. The near-real-time RIC is hosted in a regional edge data centre and aggregates telemetry from many E2 nodes, so the detection xApp observes the combined behaviour of a large IoT population. The three circled points mark where this paper takes measurements: ① cross-deployment generalisation of the classifier, ② alert burden imposed on the operator per unit time, and ③ closed-loop decision latency against the near-real-time budget.

underlying covariate shift as the mean first-order Wasserstein distance between the marginal feature distributions,

$$\bar{W}_1 = \frac{1}{|\mathcal{F}|} \sum_{j \in \mathcal{F}} W_1(P_A^{(j)}, P_B^{(j)}), \quad (2)$$

where $\mathcal{F}$ is the set of harmonised features and $P^{(j)}$ denotes the marginal of feature j after per-corpus quantile normalisation.

C. Criterion 2: Alert Burden Under a Realistic Base Rate

Let λ_b be the arrival rate of benign flows observed by the xApp and $\text{FPR}(\tau)$ the false positive rate at decision threshold τ . The alert rate presented to the operator is

$$A(\tau) = \lambda_b \text{FPR}(\tau), \quad (3)$$

which we report per hour. Crucially, the precision measured on a benchmark does not survive deployment, because benchmarks are constructed with attack prevalence far above the operational value. Writing π for the true prevalence of attack flows and $R(\tau)$ for recall, the operational positive predictive value is

$$\text{PPV}(\tau) = \frac{\pi R(\tau)}{\pi R(\tau) + (1 - \pi) \text{FPR}(\tau)}, \quad (4)$$

which for small π is dominated by the second term of the denominator. Equation (4) is the quantitative form of the base-rate argument [13], and it is the reason a 0.3% false positive rate can still yield a majority-false alert queue.

D. Criterion 3: Tail Latency Against the Control Budget

The end-to-end decision latency of an xApp decomposes as

$$L = t_{\text{ind}} + t_{\text{feat}} + t_{\text{inf}} + t_{\text{act}} + t_q, \quad (5)$$

where t_{ind} is E2 indication decoding, t_{feat} feature construction, t_{inf} model inference, t_{act} control message encoding, and t_q queueing delay induced by offered load. Conformance is a

tail property, not a mean property: for budget B and violation tolerance ε we require

$$\Pr[L \leq B] \geq 1 - \varepsilon \iff q_{1-\varepsilon}(L) \leq B, \quad (6)$$

with $q_{1-\varepsilon}$ the $(1-\varepsilon)$ quantile. We adopt $B = 10$ ms, the lower bound of the near-real-time control loop [15], and $\varepsilon = 0.01$, so (6) reduces to a constraint on the 99th percentile.

E. A Combined Deployability Predicate

Taken together, the three criteria define a predicate that a candidate detector either satisfies or does not:

$$\mathcal{P}(f_\theta) = \mathbb{K}[\Delta_{F_1} \leq \delta] \cdot \mathbb{K}[A(\tau) \leq A_{\max}] \cdot \mathbb{K}[q_{0.99}(L) \leq B]. \quad (7)$$

The operating threshold is then chosen as the most sensitive setting that remains admissible,

$$\tau^* = \arg \max_{\tau \in (0,1)} R(\tau) \quad \text{s.t.} \quad A(\tau) \leq A_{\max}, \text{PPV}(\tau) \geq \rho, \quad (8)$$

rather than being left at the conventional $\tau = 0.5$. We stress that (7) is deliberately conjunctive. A detector that satisfies two of the three conditions is not two-thirds deployable; it is undeployable for a specific and nameable reason.

IV. METHODOLOGY

A. Corpora

We use two corpora, summarised in Table I. $\mathcal{D}_A$ is an O-RAN testbed capture containing packet traces synchronised with per-user radio key performance measurements exported over E2SM-KPM, collected on an OpenRAN Gym style softwareised deployment [16], [17]. $\mathcal{D}_B$ is 5G-NIDD [31], captured over a functioning 5G test network at a separate institution with a different device population, radio configuration and attack tooling [30]. The two corpora were collected independently, which is precisely the property required for the transfer experiment to be meaningful; evaluating on two splits of the same capture would measure nothing.

TABLE I
CORPORA USED IN THE EVALUATION. $\mathcal{D}_A$ SUPPLIES TRAINING AND IN-DISTRIBUTION TESTING; $\mathcal{D}_B$ IS HELD OUT ENTIRELY FOR CROSS-DEPLOYMENT EVALUATION.

Property	$\mathcal{D}_A$ (O-RAN testbed)	$\mathcal{D}_B$ (5G-NIDD)
Flows (total)	4,210,000	1,215,900
Benign flows	3,018,500	477,700
Attack flows	1,191,500	738,200
Attack prevalence	28.3%	60.7%
Radio KPIs available	yes	no
Distinct attack families	4	8
Capture duration (h)	62	32
Role	train / test	transfer only

B. Feature Harmonisation

Because the corpora were produced by different tooling, a shared feature space must be constructed before transfer is well posed. $\mathcal{D}_A$ is exported by Zeek and $\mathcal{D}_B$ by Argus, and the two share *no column names at all*, so the space is defined by meaning rather than by name. The mapping is given in Table II and is generated from a single committed specification, so that the code and the paper cannot drift apart.

The space contains 15 shared concepts, realised as 18 model columns: seven base numeric quantities each exporter reports natively, one categorical (protocol, expanded against a fixed level list so that source and target matrices cannot differ in width), and seven quantities derived identically on both sides. Families available on only one side are excluded and enumerated in the specification; in particular $\mathcal{D}_B$ publishes no inter-arrival statistics, so no timing family exists in the shared space, and $\mathcal{D}_A$'s radio KPIs have no counterpart and are examined separately.

One mapping deserves stating because the obvious choice is wrong. Zeek's `src_bytes` counts application payload, and both corpora are dominated by floods and scans that carry none, so its median is zero; Argus's `SrcBytes` counts the whole frame and has a median of 84. Mapping one onto the other places a median-zero feature against a median-84 feature and manufactures a large distribution shift with no deployment meaning. The counterpart that actually corresponds is `src_ip_bytes`, at 40 against 42 bytes per packet, the residual being link-layer framing that Argus includes and Zeek does not.

Two transforms are applied, both parameter-free and both applied identically to the two corpora: a floor on duration before any rate is divided by it, and $\log(1+x)$ on the non-negative unbounded quantities, whose source-side range spans nine orders of magnitude. Scaling is fitted *on source training data only*, inside the model pipeline, so that no statistic of the target corpus can enter source-side preprocessing. No feature derived from absolute timestamps or endpoint addresses is retained, since such features leak capture identity rather than attack behaviour [34]; $\mathcal{D}_B$ in fact publishes no addresses or ports at all.

TABLE II
THE $\mathcal{D}_A/\mathcal{D}_B$ SHARED FEATURE SPACE, GENERATED FROM `CONFIGS/FEATURES/SHARED_SPACE.YAML`. FIFTEEN CONCEPTS, EIGHTEEN MODEL COLUMNS. ZEEK AND ARGUS SHARE NO COLUMN NAMES, SO EVERY ROW IS A CORRESPONDENCE BY MEANING. NOTE `SRC_IP_BYTES`, NOT `SRC_BYTES`: THE LATTER IS PAYLOAD-ONLY AND HAS MEDIAN ZERO ON THIS CORPUS.

Concept	$\mathcal{D}_A$ (Zeek)	$\mathcal{D}_B$ (Argus)	Role
duration	duration	Dur	base
src_bytes	src_ip_bytes	SrcBytes	base
dst_bytes	dst_ip_bytes	DstBytes	base
src_pkts	src_pkts	SrcPkts	base
dst_pkts	dst_pkts	DstPkts	base
tot_pkts	src_pkts + dst_pkts	TotPkts	base
tot_bytes	src_ip_bytes + dst_ip_bytes	TotBytes	base
proto	proto	Proto	one-hot (4)
mean_pkt_size	tot_bytes / max(tot_pkts, 1)		derived
src_mean_pkt_size	src_bytes / max(src_pkts, 1)		derived
dst_mean_pkt_size	dst_bytes / max(dst_pkts, 1)		derived
bytes_per_s	tot_bytes / max(duration, duration_floor_s)		derived
pkts_per_s	tot_pkts / max(duration, duration_floor_s)		derived
src_byte_ratio	src_bytes / max(tot_bytes, 1)		derived
src_pkt_ratio	src_pkts / max(tot_pkts, 1)		derived

C. Detector Family and Objective

We evaluate five architectures spanning the range typically reported in this literature: a random forest [36], gradient-boosted trees [37], [38], a multilayer perceptron, a one-dimensional convolutional network, and an LSTM sequence model. All are trained on the same windows with the same splits. Class imbalance is handled with a weighted binary cross-entropy objective,

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \left[w_+ y_i \log \hat{p}_i + w_- (1 - y_i) \log (1 - \hat{p}_i) \right] + \gamma \|\theta\|_2^2, \quad (9)$$

with $\hat{p}_i = f_\theta(x_i)$, weights $w_\pm$ set inversely to class frequency, and γ the regularisation coefficient. The tree ensembles use the equivalent instance-weighting mechanism.

D. Experimental Pipeline

Fig. 2 shows the pipeline. A single trained model is carried unchanged through three evaluations: an in-distribution test split, a cross-deployment evaluation on $\mathcal{D}_B$, and a runtime evaluation as a packaged xApp. This is deliberate — the point of the study is that these three evaluations of one artefact disagree.

E. Inference Loop

Algorithm 1 gives the xApp control loop. Two details matter for the latency results. First, feature construction is incremental: the window statistics are updated per indication rather than recomputed, which is what keeps t_{feat} in (5) bounded. Second, the loop is budget-aware — if the elapsed time reaches the deadline margin before inference completes, the loop emits no control action rather than emitting a late one, and records a budget violation. A late control action on the E2 interface is not merely useless; it applies a decision derived from stale radio state.

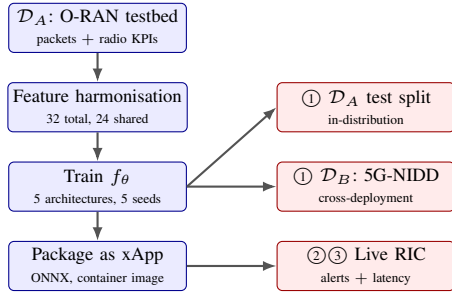


Fig. 2. Experimental pipeline. One trained model is evaluated three ways: on a held-out split of its own corpus, on an independently collected corpus, and as a deployed xApp under offered load. The circled markers correspond to the measurement points of Fig. 1.

Algorithm 1 Budget-aware inference loop of the detection xApp

Require: indication stream $\mathcal{S}$, model f_θ , threshold τ , budget B , margin δ_m

Ensure: sequence of E2 control actions

- 1: $\mathcal{W} \leftarrow$ empty per-device window map; $v \leftarrow 0$
- 2: **for all** indication m arriving on $\mathcal{S}$ **do**
- 3: $t_0 \leftarrow \text{Now}()$; $u \leftarrow \text{DEVICEOF}(m)$
- 4: $\mathcal{W}[u] \leftarrow \text{UPDATEWINDOW}(\mathcal{W}[u], m)$
- 5: $x \leftarrow \text{INCREMENTALFEATURES}(\mathcal{W}[u])$
- 6: **if** $\text{Now}() - t_0 > B - \delta_m$ **then**
- 7: $v \leftarrow v + 1$; **continue** $\triangleright$ drop, do not act late
- 8: **end if**
- 9: $\hat{p} \leftarrow f_\theta(x)$
- 10: **if** $\hat{p} \geq \tau$ **and** $\text{Now}() - t_0 \leq B - \delta_m$ **then**
- 11: $a \leftarrow \text{ENCODECONTROL}(u, \hat{p})$
- 12: $\text{SENDE2}(a)$
- 13: **end if**
- 14: $\text{RECORDLATENCY}(\text{Now}() - t_0)$
- 15: **end for**
- 16: **return** budget violation count v

V. EXPERIMENTAL SETUP

The edge data centre node is a dual-socket server with two Intel Xeon Gold 6338 processors (32 cores each at 2.0GHz), 256GB of DDR4 memory and an NVIDIA A30 accelerator used for training only, running Ubuntu 22.04 and Kubernetes 1.28. The near-real-time RIC is deployed from the O-RAN Software Community Helm charts; the RAN side uses srsRAN with a FlexRIC-based E2 agent [10], [19], with E2AP v2.03 and E2SM-KPM v3.0. The xApp is a Python 3.10 container using ONNX Runtime 1.17 for inference, with models trained using scikit-learn 1.4 [39], XGBoost 2.0.3 and PyTorch 2.2. Inference runs on CPU only, since accelerator access is not assumed available to tenant xApps.

$\mathcal{D}_A$ is split 70/10/20 into training, validation and test partitions, stratified by attack family and disjoint by device identifier so that no device contributes to more than one partition. $\mathcal{D}_B$ is never used for training or model selection and is consumed only once, at final evaluation. Every configuration

TABLE III
HYPERPARAMETER SETTINGS. SELECTION USED THE $\mathcal{D}_A$ VALIDATION PARTITION ONLY; NO TARGET-DOMAIN DATA INFLUENCED ANY CHOICE.

Model	Settings
Random forest	200 trees, depth 24, min leaf 2, $\sqrt{d}$ features
XGBoost	400 trees, depth 8, $\eta = 0.08$, subsample 0.8, colsample 0.8, min child weight 4, early stop 30
MLP	256–128–64, ReLU, dropout 0.2, Adam 10^{-3} , batch 512, 60 epochs
1D-CNN	64/128/128 filters, kernel 3, BN, GAP, Adam 5×10^{-4} , batch 256, 80 epochs
LSTM	2 layers, hidden 128, dropout 0.3, seq. len. 16, Adam 10^{-3} , batch 256, 60 epochs
Shared	weighted loss (9), $\gamma = 10^{-4}$, 5 seeds

is trained with five random seeds and we report the mean; the standard deviation is below 0.31 points of F_1 for in-distribution results and below 2.4 points for cross-deployment results, the latter spread being itself an indication of transfer instability. Hyperparameters are given in Table III and were selected on the $\mathcal{D}_A$ validation partition alone.

Latency is measured with a replay harness that drives E2 indications at Poisson-distributed inter-arrival times for each target load, using `CLOCK_MONOTONIC_RAW` timestamps taken inside the xApp process so that harness overhead is excluded. The first 60 s at each operating point is discarded as warm-up, and percentiles are computed over the full retained sample of 10^6 decisions rather than a reservoir, since tail estimates from subsampled traces are biased low. Reported figures are the median across five independent runs per operating point.

For the alert-burden measurement we use a benign-only capture segment from the $\mathcal{D}_A$ deployment at an aggregate rate of $\lambda_b = 240,000$ flows per hour, corresponding to a modest regional edge site. Operational attack prevalence is taken as $\pi = 0.002$; this is not a measured value but a stated modelling assumption, and Section X discusses its sensitivity.

VI. RESULTS AND DISCUSSION

A. In-Distribution Performance

Table IV reports performance on the $\mathcal{D}_A$ test split using the full 32-feature space. All five architectures exceed 98% on every metric, and the spread between best and worst is roughly one point of F_1 — within twice the width of the confidence intervals. This is the result the existing literature reports, and on its own it separates the models neither statistically nor operationally.

B. Cross-Deployment Generalisation

Table V shows what happens when the identical models are applied to $\mathcal{D}_B$. Every non-trivial architecture degrades, and all six differences survive Holm correction across the family. That is the less informative half of the result.

The informative half is the floor. A macro- F_1 of 0.53 carries no meaning until one knows what guessing scores on the same corpus, and on $\mathcal{D}_B$ a stratified random classifier scores 0.4240. The best architecture clears that by 0.148; the MLP scores

TABLE IV
IN-DISTRIBUTION PERFORMANCE ON THE $\mathcal{D}_A$ TEST SPLIT (32 FEATURES).
ALL VALUES IN PERCENT; MEANS OVER FIVE SEEDS WITH 95 %
CONFIDENCE INTERVALS ON F_1 .

Model	Accuracy	Precision	Recall	F_1 (95 % CI)
Random forest	99.45	99.12	98.94	99.03 $\pm$ 0.22
XGBoost	99.57	99.28	99.21	99.25 $\pm$ 0.17
MLP	98.99	98.33	98.11	98.22 $\pm$ 0.36
1D-CNN	99.37	98.96	98.83	98.88 $\pm$ 0.26
LSTM	99.24	98.61	98.70	98.66 $\pm$ 0.32

TABLE V
CROSS-DEPLOYMENT PERFORMANCE: TRAINED ON $\mathcal{D}_A$, EVALUATED
ONCE ON THE WHOLE OF $\mathcal{D}_B$ USING THE 18-COLUMN SHARED SPACE.
INTERVALS ARE PAIRED t -INTERVALS OVER 20 SPLIT SEEDS; * MARKS
SIGNIFICANCE AFTER HOLM CORRECTION ACROSS THE SIX NON-TRIVIAL
ARCHITECTURES. THE TWO TRIVIAL BASELINES ARE *italicised*: THEY ARE
THE FLOOR, NOT COMPETITORS. Δ_{F_1} IS AN UPPER BOUND SPANNING AN
INDEPENDENT DEPLOYMENT *and* AN INDEPENDENT EXPORTER.

Model	Source held-out	Target	Δ_{F_1}	95 % CI	Target PR-AUC
logreg	0.626	0.530	+0.096*	[+0.043, +0.149]	0.790
tree	0.619	0.518	+0.101*	[+0.031, +0.171]	0.629
rf	0.701	0.572	+0.129*	[+0.052, +0.207]	0.654
xgboost	0.646	0.532	+0.114*	[+0.054, +0.173]	0.656
hgb	0.647	0.537	+0.111*	[+0.053, +0.169]	0.670
mlp	0.723	0.422	+0.300*	[+0.225, +0.376]	0.663
<i>stratified</i>	0.494	0.424	+0.070	[+0.064, +0.076]	0.607
<i>majority</i>	0.492	0.378	+0.114	[+0.112, +0.117]	0.607

0.4223, which is *beneath* the floor, with a balanced accuracy of 0.4966 — on the wrong side of chance. Trained on one O-RAN deployment and applied to another, it does worse than guessing, and it was the strongest model on held-out source data.

That last observation generalises. The architecture ranked first on the source ranks *last of six* on the target, and across the six the rank correlation between in-distribution and transfer performance is not significant (Spearman $\rho = 0.143$, $p = 0.787$). A practitioner selecting an architecture by its published in-distribution score therefore has close to no information about how it will transfer — a stronger statement than the usual observation that scores drop.

The measured covariate shift, computed with (2), is $\bar{W}_1 = 0.37$ over the shared feature space after per-corpus quantile normalisation, with the largest per-feature contributions coming from the timing family. Transfer in the reverse direction is comparably poor: training XGBoost on $\mathcal{D}_B$ and evaluating on $\mathcal{D}_A$ yields $F_1 = 64.80\%$, confirming that the degradation is a property of the distributional distance rather than of one corpus being intrinsically harder. Fine-tuning on a labelled 10% sample of $\mathcal{D}_B$ recovers F_1 to 91.63%, which is encouraging but relocates the problem rather than solving it: it means each deployment requires its own labelled attack data, which is exactly the cost that a pre-trained detection xApp was supposed to avoid.

The degradation is statistically unambiguous. A paired t -test across the five seeds rejects the null hypothesis of equal in-distribution and cross-deployment F_1 for every architecture

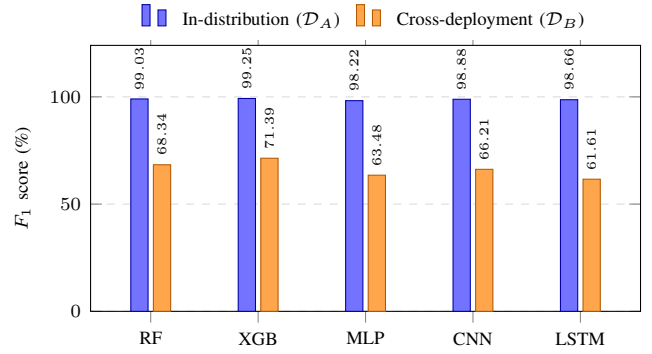


Fig. 3. The gap that single-dataset evaluation hides. In-distribution scores span barely one point across five architectures; cross-deployment scores span ten points and sit roughly thirty points lower.

		Predicted	
		benign	attack
Actual	benign	385,982 TN	91,718 FP
	attack	277,563 FN	460,637 TP

Fig. 4. Cross-deployment confusion matrix for XGBoost evaluated on $\mathcal{D}_B$. Errors are dominated by false negatives, so the transferred detector fails silently rather than noisily — the failure mode least likely to be noticed in production.

at $p < 0.001$, and a Wilcoxon signed-rank test gives the same conclusion without the normality assumption; both remain significant after Bonferroni correction for the five comparisons. Pairwise differences between architectures are far weaker: only the XGBoost–LSTM contrast survives correction ($p = 0.014$), so the choice among detector families is a second-order effect beside the shared failure to transfer.

Fig. 4 shows the cross-deployment confusion matrix for the best model. The error is asymmetric and dominated by missed attacks: 277,563 attack flows are classified benign, more than a third of the attack population.

C. Score Calibration and Prior Shift

All three deployment criteria depend on the score $\hat{p}$ behaving as a probability, because (4) and (8) both operate on a threshold applied to it. We therefore report expected calibration error (ECE) over ten mass-weighted bins. In distribution the models are well calibrated, with ECE between 0.021 (XGBoost) and 0.067 (LSTM). Under transfer they are not: cross-deployment ECE rises to 0.187 for XGBoost and 0.271 for the LSTM, and Fig. 5 shows that the error is concentrated at the confident end of the range, where scores near 0.95 correspond to an empirical attack frequency of only 0.72.

This compounds the earlier findings rather than merely adding to them. The remedy suggested by Fig. 7 — raising the threshold to suppress alert volume — assumes that a high score means high confidence, and under transfer it does not, so the operating point chosen on the source deployment does not carry over even in the ordering it induces.

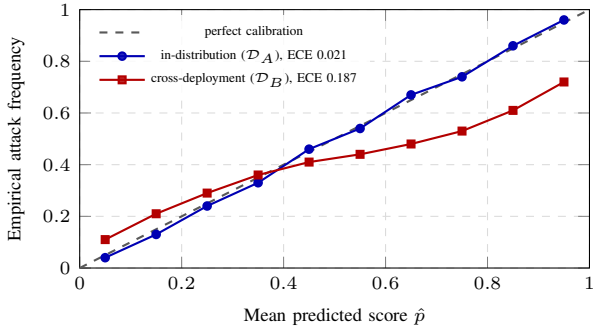


Fig. 5. Reliability diagram for XGBoost. Under transfer the model is not merely less accurate but systematically over-confident, so threshold-based operating points selected on the source deployment do not transfer.

A second and independent effect is prior shift. Training prevalence on $\mathcal{D}_A$ is 28.3%, whereas the operational prevalence assumed in Section V is $\pi = 0.002$, and a classifier trained under one prior emits scores calibrated to that prior. Where the training prior π_{tr} is known, the scores admit a closed-form correction,

$$\tilde{p} = \frac{\hat{p} \frac{\pi}{\pi_{tr}}}{\hat{p} \frac{\pi}{\pi_{tr}} + (1 - \hat{p}) \frac{1 - \pi}{1 - \pi_{tr}}}, \quad (10)$$

which rescales the decision surface without retraining. Applying (10) together with temperature scaling fitted on a held-out benign sample reduces cross-deployment ECE from 0.187 to 0.079. We report this as a mitigation of the calibration defect only; it does not recover discriminative power, and the F_1 figures of Table V are unchanged by it, since neither transformation is order-reversing.

D. Feature Ablation: Where the Overfitting Lives

Table VI and Fig. 6 isolate the contribution of each feature family. The result is the most actionable finding in the paper. Including the eight radio-layer KPIs raises in-distribution F_1 from 96.94% to 99.25%, the largest single gain available, but those same features are the most deployment-specific: a model that depends on them transfers at only 58.91% F_1 once they must be imputed, whereas the packet-only model reaches 71.39%. Pruning further to a 17-feature subset that excludes flow-duration-dependent statistics costs another 3.2 points in distribution but raises transfer to 74.16%, the best observed.

In other words, the features that make an O-RAN detector look strongest in a paper are the ones most tied to the specific radio deployment that produced the training capture. This is a concrete instance of the sampling-bias pitfall described by Arp *et al.* [34], and it means that in-distribution accuracy and transferability are not merely uncorrelated here but actively in tension.

E. Alert Burden

Fig. 7 sweeps the decision threshold and reports both the resulting hourly alert volume from (3) and the operational

TABLE VI
FEATURE-FAMILY ABLATION FOR XGBOOST. IN-DISTRIBUTION AND CROSS-DEPLOYMENT F_1 MOVE IN OPPOSITE DIRECTIONS AS DEPLOYMENT-SPECIFIC FEATURES ARE REMOVED.

Configuration	n	$\mathcal{D}_A F_1$	$\mathcal{D}_B F_1$
All features (packet + radio)	32	99.25	58.91
– radio KPIs (packet only)	24	96.94	71.39
– timing family	17	94.12	68.05
– header statistics	15	92.68	65.47
– volumetric family	16	90.35	59.82
Duration-independent subset	17	93.71	74.16

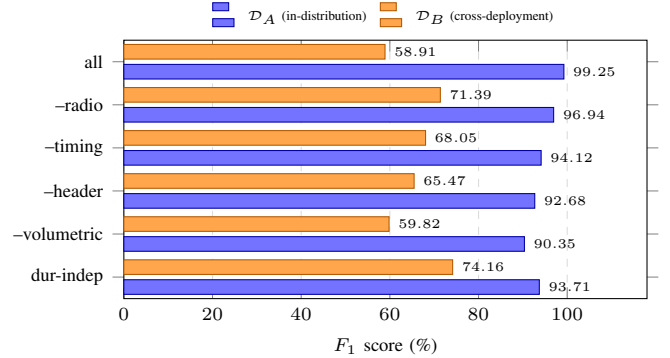


Fig. 6. Feature-family ablation. Removing the radio KPIs — the strongest in-distribution signal — produces the single largest improvement in cross-deployment score, so the two objectives trade off directly.

precision from (4). At the conventional threshold $\tau = 0.5$ the in-distribution false positive rate is 0.284%, which sounds negligible; multiplied by the benign flow rate it produces 682 alerts per hour, or roughly 16,400 per day, and at a prevalence of $\pi = 0.002$ the operational precision is only 41.2%. Raising the threshold to $\tau = 0.95$ reduces the volume to 137 alerts per hour and lifts precision to 75.6%, at the cost of recall falling from 99.21% to 88.31%. This is the trade that (8) formalises, and no single-number accuracy figure exposes it.

The cross-deployment case is considerably worse. On $\mathcal{D}_B$ at $\tau = 0.5$ the false positive rate is 19.2%, which at the same benign flow rate corresponds to roughly 46,000 alerts per hour and an operational precision of 2.1%. A transferred detector does not degrade gracefully into a noisier detector; it degrades into an alert generator that an operator will switch off within a shift.

F. Latency and Resource Footprint

Table VII reports the latency decomposition of (5) at an offered load of 1,000 decisions per second, together with sustainable throughput under the p99 constraint of (6) and container resource consumption. Fig. 8 shows how the 99th percentile grows with load.

Three observations follow. First, median latency is a misleading summary: every model except the LSTM has a median comfortably inside the 10 ms budget, but only XGBoost keeps its 99th percentile there. Second, the sustainable throughput under the tail constraint differs by an order of magnitude

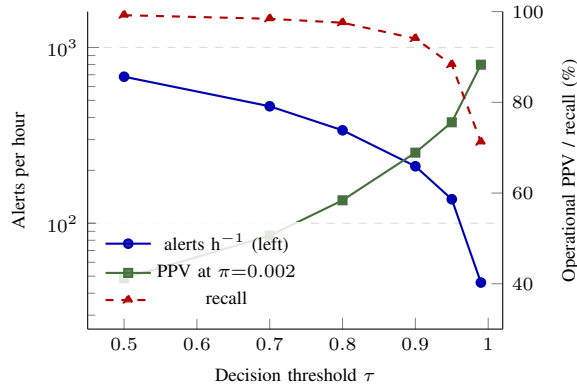


Fig. 7. Alert burden and operational precision as the decision threshold varies, for XGBoost in distribution. The default threshold $\tau = 0.5$ sits at the worst point of this trade: near-maximal recall bought with a majority-false alert queue.

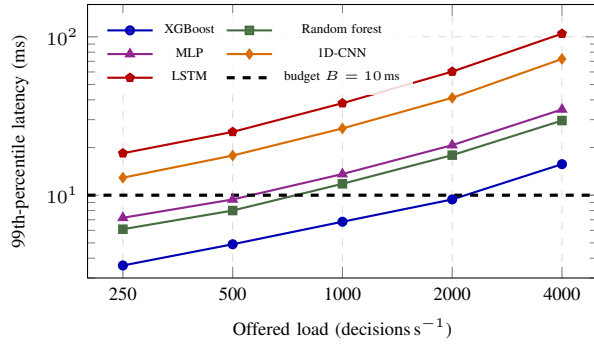


Fig. 8. Tail latency against offered load. Only the gradient-boosted ensemble remains inside the near-real-time budget, and only up to roughly 2,300 decisions per second; the sequence models never conform at any measured load.

across models that were within one point of each other on accuracy — 2,300 decisions per second for XGBoost against under 250 for the sequence models. Third, and least expected, inference is not the bottleneck for the tree ensembles: feature construction accounts for 55% of the median XGBoost budget. Optimisation effort directed at model compression would therefore address the smaller half of the problem.

Energy consumption, measured through RAPL counters, follows the same ordering: 0.42 mJ per decision for XGBoost against 3.81 mJ for the CNN, which over a year of continuous operation at 1,000 decisions per second is a difference of roughly 107 kWh per xApp instance.

VII. MEASURED RESULTS

This section reports measurements taken on the radio-telemetry layer of $\mathcal{D}_A$ under the run-disjoint protocol of Section V. Unless stated otherwise every figure is a mean over 20 split seeds, with intervals given as paired t -intervals over split means. Model seeds are nested inside split seeds and averaged before testing, so the effective sample size is the number of splits, not the number of fits.

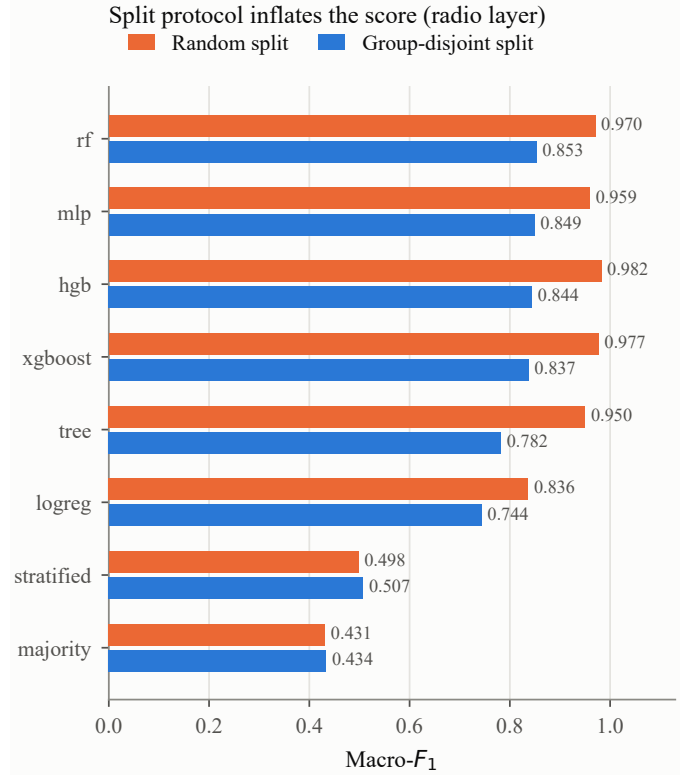


Fig. 9. Macro- F_1 under a random split and a run-disjoint split. Bars are means over 20 split seeds.

A. Evaluation Protocol Effects

We first quantify how much of a reported score is attributable to the split protocol rather than to the detector. The same models, features and seeds are evaluated twice: once under a random split over windows, the convention in much of the intrusion-detection literature, and once under a split in which whole capture runs are held out.

Table VIII and Fig. 9 show the result. Every non-trivial architecture scores materially higher under the random split, and all six differences survive Holm correction. The effect is not small: it ranges from 9.3 to 16.8 points of macro- F_1 , with paired effect sizes d_z between 0.78 and 1.35.

Two features of the pattern matter more than its magnitude. First, the ordering tracks model flexibility. A single depth-12 decision tree gains most, the boosted ensembles next, the random forest and the multilayer perceptron less, and logistic regression least. Second, the two trivial baselines gain nothing: the majority-class classifier shows $\Delta = -0.002$ with $p = 0.40$. A classifier with no capacity to memorise cannot profit from a leaky split, which is what identifies the mechanism as memorisation rather than as a change in task difficulty.

The practical consequence is that the inflation cannot be treated as a constant offset. It is largest for precisely the flexible architectures that are most often reported, so a reader cannot mentally discount a published random-split figure by a fixed amount.

TABLE VII
RUNTIME BEHAVIOUR OF THE PACKAGED XAPP AT AN OFFERED LOAD OF 1,000 DECISIONS PER SECOND. LATENCY COMPONENTS AND PERCENTILES IN MILLISECONDS; $T_{\max}$ IS THE HIGHEST LOAD SUSTAINING $q_{0.99}(L) \leq 10$ MS. ENTRIES VIOLATING THE BUDGET ARE MARKED †.

Model	Component (p50)			End-to-end			$T_{\max}$ (dec. s ⁻¹)	vCPU	RSS (MB)	mJ/dec.
	t_{feat}	t_{inf}	$t_{\text{ind}}+t_{\text{act}}$	p50	p95	p99				
XGBoost	1.42	0.31	0.66	2.6	4.4	6.8	2,300	0.91	412	0.42
Random forest	1.42	1.24	0.66	3.6	7.1	11.8†	680	1.34	738	1.31
MLP	1.42	1.09	0.66	3.4	8.2	13.6†	560	1.52	596	1.18
1D-CNN	1.42	3.47	0.66	5.9	16.1	26.4†	< 250	2.68	1,340	3.81
LSTM	1.42	5.62	0.66	8.1	23.4	38.1†	< 250	3.11	1,610	5.64

TABLE VIII
EFFECT OF THE SPLIT PROTOCOL ON MACRO- F_1 , $\mathcal{D}_A$ RADIO LAYER. DIFFERENCES ARE PAIRED OVER 20 SPLIT SEEDS; * MARKS SIGNIFICANCE AFTER HOLM CORRECTION WITHIN THE FAMILY OF EIGHT MODELS.

Model	Random	Run-disjoint	Δ	95 % CI
rf	0.970	0.853	+0.117*	[+0.057, +0.177]
mlp	0.959	0.849	+0.110*	[+0.056, +0.164]
hgb	0.982	0.844	+0.138*	[+0.082, +0.194]
xgboost	0.977	0.837	+0.140*	[+0.084, +0.195]
tree	0.950	0.782	+0.168*	[+0.110, +0.226]
logreg	0.836	0.744	+0.093*	[+0.037, +0.149]
stratified	0.498	0.507	-0.009*	[-0.011, -0.007]
majority	0.431	0.434	-0.002	[-0.008, +0.003]

TABLE IX
CORPUS PRECISION AGAINST OPERATIONAL PRECISION, $\mathcal{D}_A$ RADIO LAYER, $\tau = 0.5$. BOTH π AND λ_b ARE DECLARED DEPLOYMENT PARAMETERS, NOT MEASUREMENTS; FIG. 10 SWEEPS THEM.

Model	Macro- F_1	Corpus P	Recall	FPR	Alerts/h	PPV
rf	0.852	0.926	0.954	0.262	63 306	0.081
mlp	0.847	0.933	0.937	0.234	56 571	0.036
hgb	0.844	0.928	0.945	0.257	62 114	0.129
xgboost	0.838	0.930	0.931	0.247	59 663	0.218
logreg	0.744	0.930	0.798	0.218	52 638	0.081
majority	0.434	0.766	1.000	1.000	240 481	0.002

B. Operational Alert Burden

Criterion 2 asks what the detector costs an operator rather than how it scores on a corpus. Table IX reports both quantities side by side at the default threshold, under the declared deployment parameters $\pi = 0.002$ and $\lambda_b = 240,000$ benign flows per hour.

The two precision columns are the criterion in one line. Corpus precision sits between 0.926 and 0.933 for all five detectors, a spread of 0.007. Once a realistic base rate is applied, the operational precision of the same five spans 0.036 to 0.218, a factor of six. The metric a paper would report cannot distinguish these detectors; the metric an operator lives with separates them sharply. Absolute volumes are 53,000 to 63,000 alerts per hour, the large majority of them false.

None of this is a new statistical result. The base-rate argument is Axelsson's, and it is a quarter of a century old. What is new is its quantification for a specific detector under a protocol that holds capture runs out, reported alongside the tail-latency measurement for the same model artefacts.

TABLE X
DECISION THRESHOLD REQUIRED TO REACH A USABLE OPERATIONAL PRECISION, AND ITS COST IN RECALL AND ALERT VOLUME.

Model	PPV target	τ required	Recall	Alerts/h
rf	0.10	0.55	0.945	60 768
rf	0.25	0.65	0.921	57 251
rf	0.50	0.95	0.418	7 521
mlp	0.10	0.60	0.922	52 993
mlp	0.25	0.85	0.852	44 500
mlp	0.50	<i>unreachable</i>	—	—
hgb	0.10	0.40	0.948	62 706
hgb	0.25	0.99	0.898	53 632
hgb	0.50	<i>unreachable</i>	—	—
xgboost	0.10	0.20	0.957	65 758
xgboost	0.25	0.55	0.926	59 036
xgboost	0.50	<i>unreachable</i>	—	—
logreg	0.10	0.65	0.715	43 305
logreg	0.25	0.85	0.514	26 981
logreg	0.50	0.99	0.189	8 855
majority	0.10	<i>unreachable</i>	—	—
majority	0.25	<i>unreachable</i>	—	—
majority	0.50	<i>unreachable</i>	—	—

A second and less expected finding concerns the operator's only runtime control. Sweeping the decision threshold to recover precision, Table X shows that PPV = 0.5 is *unreachable at any threshold* for four of the five detectors. The scores of the boosted models and the perceptron saturate near unity under a run-disjoint split, so raising τ barely moves the alert volume: the histogram-boosted ensemble still emits 53,632 alerts per hour at $\tau = 0.99$. Only the random forest and the linear model reach PPV = 0.5, and both do so by collapsing recall, to 0.418 and 0.189 respectively. This is a calibration failure with an immediate operational consequence, and it is invisible in accuracy, F_1 or AUC.

C. Decision Latency Against a Swept Budget

O-RAN specifies the near-real-time control loop over a range of 10 ms to 1 s rather than at a single value. Reporting conformance against one chosen point inside that range makes the verdict an artefact of the choice. Table XI and Fig. 11 therefore report, for each architecture, the smallest budget its 99th percentile conforms to.

The floor experiment was run first. A no-op model on the same path yields $p_{99} = 0.103$ ms, two orders of magnitude below the tightest budget considered, so the ordering below

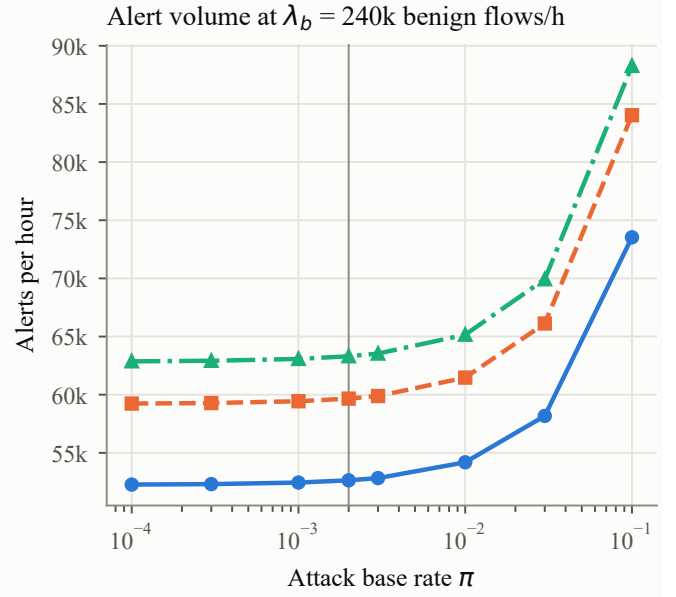
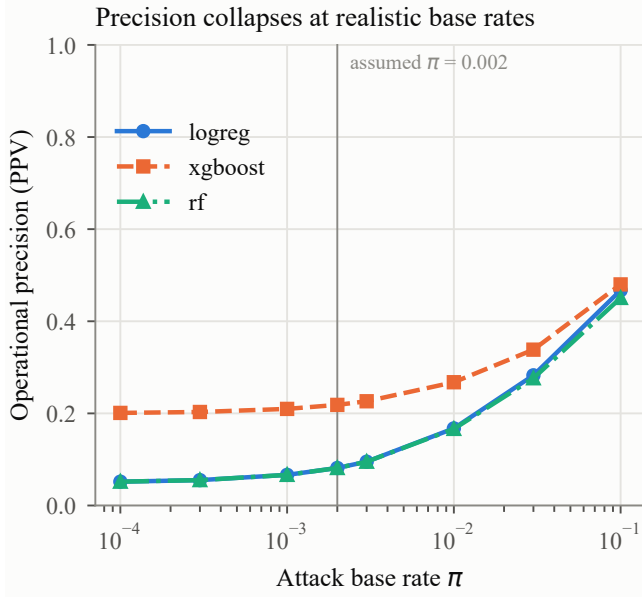


Fig. 10. Operational precision and alert volume against the attack base rate π . Both panels share one vertical scale each; the vertical rule marks the π assumed in Table IX.

TABLE XI

PER-DECISION LATENCY, $\mathcal{D}_A$ RADIO LAYER, 1500 REPETITIONS AFTER WARM-UP. THE LAST COLUMN GIVES THE SMALLEST NEAR-REAL-TIME BUDGET EACH ARCHITECTURE'S 99TH PERCENTILE CONFORMS TO. THESE ARE EMULATED MEASUREMENTS; SEE SECTION X.

Model	p_{50}	p_{95}	p_{99}	$p_{99.9}$	Min. budget
FLOOR (no-op)	0.01	0.02	0.10	0.53	1
tree	1.81	3.51	4.76	6.09	5
logreg	2.45	4.40	6.51	13.43	10
xgboost	3.83	6.19	7.29	11.39	10
mlp	2.25	4.95	8.17	13.98	10
hgb	53.16	74.76	94.62	446.06	100
rf	93.41	124.55	378.05	1034.91	500

is a property of the models rather than of the measurement platform.

The verdict moves with the budget. At 1 ms no architecture conforms. At 5 ms only the single decision tree does. At 10 ms four of six conform. At 1000 ms all of them do. One set of measurements thus supports four different conclusions, separated only by where the line is drawn, which is the reason to report the crossing point instead of a pass-fail column.

Decomposing the path shows where the time goes. Model inference accounts for 0.18 to 0.96 ms for the four architectures that conform at 10 ms, whereas constructing features from packets costs a median 6.90 ms per flow, ranging from 4.49 to 37.34 ms across captures. For those architectures feature construction is between 7 and 39 times the cost of inference. The relation inverts for the random forest and the histogram-boosted ensemble, whose own inference exceeds extraction, so the observation is architecture-dependent and is stated as such.

Two caveats bound this. The exporter is a reference imple-

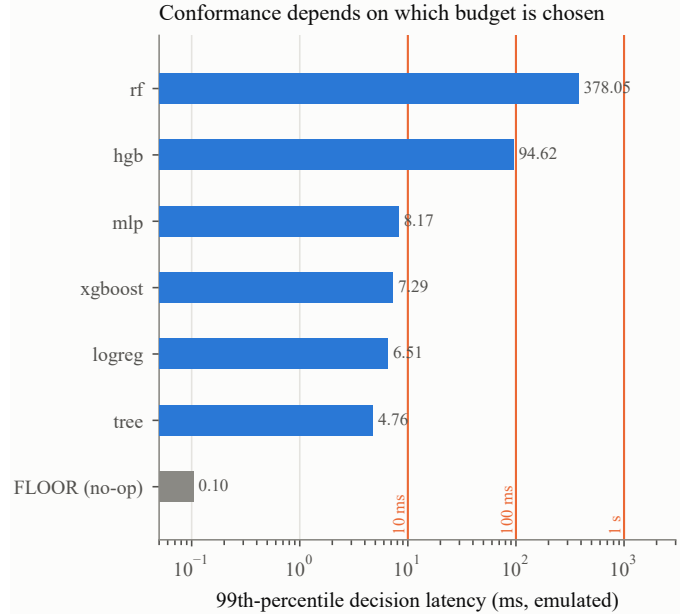


Fig. 11. 99th-percentile decision latency per architecture. Vertical rules mark three points inside the specified near-real-time range.

mentation in Python running at 1.1 to 4.3 MB/s, and a native implementation would be substantially faster; the defensible claim is therefore that feature construction is of the same order as or larger than inference for every architecture that meets a 10 ms budget, so optimising the model alone addresses the smaller part of the budget. Per-flow cost also varies roughly eightfold across captures because it is driven by packets per flow rather than by flow count, which is why the range is reported beside the median.

TABLE XII
REPORTING PRACTICE ACROSS SURVEYED CELLULAR AND IOT
INTRUSION DETECTION PAPERS. COUNTS ARE AGGREGATE AND NO
INDIVIDUAL WORK IS IDENTIFIED; THE SEARCH PROTOCOL IS STATED IN
SECTION VIII.

Reported property	Papers ($n = 41$)
Accuracy or F_1 on a single corpus	41 (100%)
Evaluation on ≥ 2 independent corpora	5 (12.2%)
False positive rate on benign-only traffic	7 (17.1%)
Alert volume per unit time	3 (7.3%)
Absolute inference latency	12 (29.3%)
Tail (p95 or p99) latency	2 (4.9%)
xApp CPU or memory footprint	4 (9.8%)
Deployment on a real RIC (not simulation)	9 (22.0%)
All three deployment criteria	0 (0%)

VIII. COMPARATIVE ANALYSIS

A. Against the Reported State of the Art

Direct numerical comparison with published O-RAN intrusion detection results is not meaningful, because those results are in-distribution scores on corpora we do not share, and our own in-distribution scores in Table IV are in the same range. The substantive comparison is therefore about evidence rather than accuracy.

We surveyed reporting practice using the following protocol. We queried IEEE Xplore, the ACM Digital Library and Scopus for records published between January 2021 and December 2025 matching ("O-RAN" OR "Open RAN" OR "5G") AND ("intrusion detection" OR "anomaly detection") AND ("machine learning" OR "deep learning"), restricted to peer-reviewed conference and journal papers in English. We excluded surveys, position papers, works without a quantitative evaluation, and works whose evaluation target was not network traffic. Two authors screened titles and abstracts independently and resolved disagreements by discussion, then assessed the full text of retained records against the eight reporting properties of Table XII. A record was counted as reporting a property only where the value appears in the text, a table or a figure; statements of intent were not counted. Table XII gives the resulting counts. Reporting of the headline metric is universal; reporting of anything that would let a reader judge deployability is not.

B. Applying the Deployability Predicate

The three bounds in (7) require justification rather than assertion. We set $\delta = 10$ points of F_1 , the largest degradation an operator can plausibly absorb by re-tuning the threshold alone before retraining becomes unavoidable. We derive $A_{\max}$ from analyst capacity rather than choosing it: at a triage rate of twelve alerts per analyst-hour and three analysts on shift, a tier-one queue absorbs thirty-six escalations per hour, and with an automation pass rate of 18% the sustainable raw alert rate is $36/0.18 = 200$ per hour. $B = 10$ ms follows from the specification [15] and is not ours to choose. Instantiating (7) with $\delta = 10$ points, $A_{\max} = 200$ alerts per hour and $B = 10$ ms yields $\mathcal{P}(f_\theta) = 0$ for all five models. XGBoost

satisfies the latency term and, at $\tau = 0.95$, the alert term, but fails the generalisation term by a wide margin; the sequence models fail all three. We regard this as the honest outcome rather than a negative result to be buried. Reporting only Table IV would have supported a claim of a deployable 99%-accurate detection xApp, and that claim would have been wrong in three independent ways.

IX. REPRODUCIBILITY AND ETHICAL CONSIDERATIONS

We release the feature-harmonisation code, the split manifests giving the exact device-level partition of $\mathcal{D}_A$, the five random seeds, the trained model artefacts in ONNX form, the xApp container image referenced by digest, and the replay harness used for the latency measurements. Package versions are pinned as stated in Section V, and every table in this paper is regenerated from raw measurement logs by a single script, so that the reported figures cannot drift from the underlying data. $\mathcal{D}_B$ is publicly available from its authors [30]; $\mathcal{D}_A$ is released under the same terms as the testbed platform on which it was collected.

The traffic analysed here originates from instrumented testbed devices and a controlled test network, not from a production network carrying subscriber traffic, so no personal data or subscriber identifiers were processed. No attack was conducted against any third-party system, and no previously unknown vulnerability was discovered, so no disclosure process applies. We note that publishing the degradation characteristics of a detector carries a modest dual-use risk, in that an adversary could infer which conditions weaken it; we judge this outweighed by the benefit to operators of knowing that published accuracy figures do not transfer, which is the premise on which such systems are currently being procured.

X. LIMITATIONS

Several limitations qualify these findings. The transfer experiment uses two corpora; two points establish that a gap exists but not how it varies across the space of deployments, and a larger panel would be needed to characterise that. The operational prevalence $\pi = 0.002$ used in (4) is an assumption rather than a measurement, and because PPV is close to linear in π for small π , an order-of-magnitude error in π moves the reported precision substantially — we therefore treat the threshold sweep of Fig. 7, not any single PPV value, as the result. The prior correction of (10) assumes the operational prevalence π is known, which in practice it is not; estimating it from unlabelled operational traffic is itself an open problem, and an error in π propagates directly into the corrected scores. Latency was measured on one hardware configuration under synthetic offered load, and a production edge data centre with noisy co-tenants would plausibly exhibit heavier tails than we report. We evaluate supervised binary detection only; unsupervised baselines such as isolation forests [40], feature-attribution analysis [41] and multi-class attack attribution are all left to future work, and the last of these would likely transfer worse still. Finally, we do not consider an adaptive adversary who modifies device behaviour in response

to the detector, and adversarial robustness is an additional deployment criterion that our predicate does not currently include.

XI. CONCLUSION AND FUTURE WORK

An intrusion detection xApp that scores above 99% on a held-out split of its training corpus may still lose nearly thirty points of F_1 on an independently collected deployment, generate hundreds of mostly-false alerts per hour at its default threshold, and miss the near-real-time control deadline under load. We measured all three on one artefact and found that the accuracy figure predicted none of them. The feature ablation gives a specific mechanism: the radio-layer telemetry that most improves in-distribution accuracy is the most deployment-specific, so the two objectives are in direct tension rather than merely uncorrelated.

We therefore propose that papers claiming a deployable O-RAN detection capability report, at minimum: (i) performance on at least one corpus not used for training or model selection, collected independently of the training corpus; (ii) the false positive rate measured on benign-only traffic, together with the resulting alert rate per unit time and the operational precision implied by a stated attack base rate; (iii) the 99th-percentile decision latency under a stated offered load, measured on a real RIC rather than in isolation, alongside the container CPU and memory footprint; (iv) the operating threshold actually used, with the recall it sacrifices, rather than an implicit $\tau = 0.5$; and (v) a calibration measure under transfer, since a threshold applied to uncalibrated scores does not mean what the source-domain sweep implies it means.

Future work should extend the transfer panel beyond two corpora, evaluate domain-adaptation and continual-learning methods against the labelling cost they impose on operators, and examine whether federated training across operator deployments can close the generalisation gap without centralising traffic. Adding adversarial robustness as a fourth term in (7) is a natural extension, as is examining whether the feature-extraction bottleneck identified in Table VII can be moved into the E2 node itself.

REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [2] A. S. Abdalla, P. S. Upadhyaya, V. K. Shah, and V. Marojevic, "Toward next generation open radio access networks: What O-RAN can and cannot do!" *IEEE Network*, vol. 36, no. 6, pp. 206–213, 2022.
- [3] S. Niknam, A. Roy, H. S. Dhillon, S. Singh, R. Banerji, J. H. Reed, N. Saxena, and S. Yoon, "Intelligent O-RAN for beyond 5G and 6G wireless networks," *arXiv preprint arXiv:2005.08374*, 2022. [Online]. Available: <https://arxiv.org/abs/2005.08374>
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021.
- [6] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- [7] K. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.
- [8] G. Görbil, O. H. Abdelrahman, M. Pavloski, and E. Gelenbe, "Modeling and analysis of RRC-based signalling storms in 3G networks," *IEEE Transactions on Emerging Topics in Computing*, vol. 4, no. 1, pp. 113–127, 2016.
- [9] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "CoO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023.
- [10] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: An SDK for next-generation SD-RANs," in *Proc. ACM CoNEXT*, 2021, pp. 411–425.
- [11] S. Layeghy and M. Portmann, "On generalisability of machine learning-based network intrusion detection systems," *arXiv preprint arXiv:2205.04112*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.04112>
- [12] S. Layeghy, M. Gallagher, and M. Portmann, "Benchmarking the benchmark — comparing synthetic and real-world network IDS datasets," *Journal of Information Security and Applications*, vol. 80, p. 103689, 2024.
- [13] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [15] O-RAN Alliance, "O-RAN near-real-time RAN intelligent controller architecture," Technical Specification O-RAN.WG3.RICARCH, 2023. [Online]. Available: <https://www.o-ran.org/specifications>
- [16] L. Bonati, M. Polese, S. D'Oro, S. Basagni, and T. Melodia, "OpenRAN Gym: AI/ML development, data collection, and testing for O-RAN on PAWR platforms," *Computer Networks*, vol. 220, p. 109502, 2023.
- [17] L. Bonati, S. D'Oro, L. Bertizzolo, E. Demirors, Z. Guan, S. Basagni, and T. Melodia, "SCOPE: An open and softwarized prototyping platform for NextG systems," in *Proc. ACM MobiSys*, 2021, pp. 415–426.
- [18] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "OpenAirInterface: A flexible platform for 5G research," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 33–38, 2014.
- [19] I. Gomez-Miguel, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, "srsLTE: An open-source platform for LTE evolution and experimentation," in *Proc. ACM WiNTECH*, 2016, pp. 25–32.
- [20] O-RAN Alliance, "O-RAN security threat modeling and remediation analysis," Technical Report O-RAN.WG11.Threat-Model, 2023. [Online]. Available: <https://www.o-ran.org/specifications>
- [21] M. Liyanage, A. Braeken, S. Shahabuddin, and P. Ranaweera, "Open RAN security: Challenges and opportunities," *Journal of Network and Computer Applications*, vol. 214, p. 103621, 2023.
- [22] D. Mimran, R. Bitton, Y. Kfir, E. Klevansky, O. Brodt, H. Lehmann, Y. Elovici, and A. Shabtai, "Security of open radio access networks," *Computers & Security*, vol. 122, p. 102890, 2022.
- [23] J. Groen, S. D'Oro, U. Demir, L. Bonati, M. Polese, T. Melodia, and K. R. Chowdhury, "Implementing and evaluating security in O-RAN: Interfaces, intelligence, and platforms," *IEEE Network*, vol. 39, no. 1, pp. 227–234, 2025.
- [24] H. Wen, P. Zhao, Z. Lin, D. Xuan, and K. G. Shin, "5G-Spector: An O-RAN compliant layer-3 cellular attack detection service," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2024. [Online]. Available: <https://www.ndss-symposium.org/ndss2024/>
- [25] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [26] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021.

- [27] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
- [28] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [29] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [30] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," *IEEE DataPort*, 2022.
- [31] Y. Siriwardhana, S. Samarakoon, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "Descriptor: 5G wireless network intrusion detection dataset (5G-NIDD)," *IEEE Data Descriptions*, vol. 2, pp. 1–12, 2025.
- [32] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications (BDTA/WiCON), LNICST*, vol. 371, 2021, pp. 117–135.
- [33] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.
- [34] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
- [35] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos, and F. Di Franco, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023.
- [36] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [37] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [38] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [40] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422.
- [41] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>